A SHORT LECTURE IN SIX PARTS

What is Computer Science?

Christoph Kirsch
University of Salzburg, Austria
Czech Technical University in Prague, Czechia

Everything that follows is built from one distinction: the difference between a proof and the truth.

Those two colours keep those two jobs for the whole half hour: proof, notation, syntax — truth, meaning, semantics.

FULL DISCLOSURE

This entire talk was generated by an AI.

It took me 40+ years to become someone who could prompt it into existence. I could do that today. I could not have done it yesterday. No prior computer science is required to follow what it produced.

THE QUESTION

The name misleads twice. It is not about computers, and it is only sometimes a science.

What it studies is older than both: notation — what can be written down, and what the marks can and cannot do once they are written.

It met that question where a machine forces the notation to be exact. That is why the limits were noticed here first, and measured here most sharply — and why they hold everywhere else.

So the answer to the title is a definition, stated at the end and earned first. Nothing in the derivation depends on which computer, which language, or which year.

> *Computer science is no more about computers than astronomy is about telescopes.*
>
> ATTRIBUTED TO EDSGER W. DIJKSTRA

The short answer, to be earned: computer science is the exact study of the gap between notation and meaning.

ROUTE

Six parts.

I	Size	why 34 bytes beat the universe
II	Infinity	why meanings outnumber notations
III	Self-reference	why proof ≠ truth · and where the computer came from
IV	Cost	hard to find, easy to check
V	Machines	a language model, seen from inside the theorems
VI	Definition	what computer science is · and a specimen to run

Four of these parts end in a theorem that sounds like bad news. Each one turns out to mark the place where the subject begins.

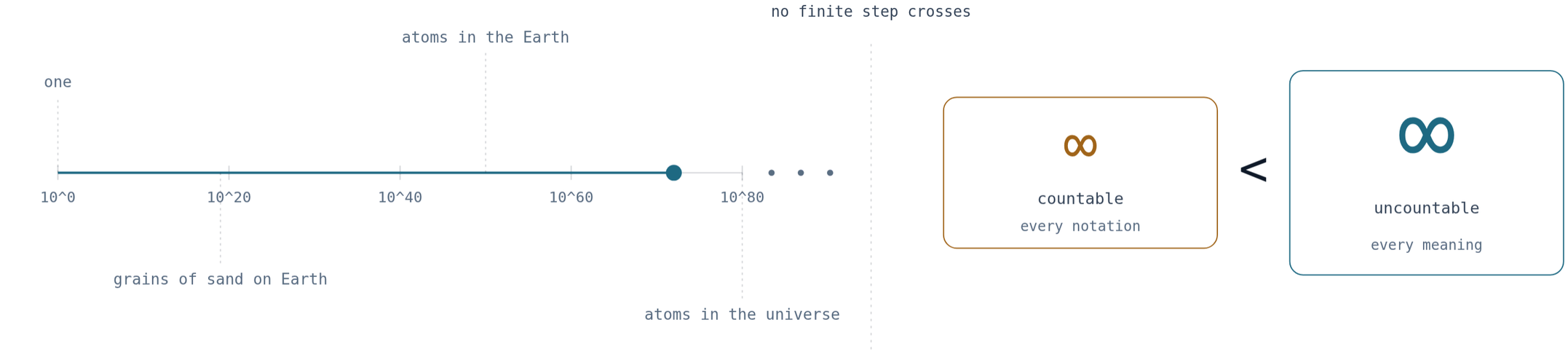
PART I

Size

First a feeling for how big the spaces are that programs live in — because everything later, including every bug and every AI, happens inside one.

THE MAP

One line carries the whole talk: small, vast, then two sizes of endless.



Left. From one thing to every atom there is — the whole physical world in eighty steps, each one ten times the last.

The wall. No number of steps crosses it. Infinity is not the far end of the line; it is what the line never reaches.

Right. Past the wall, endlessness comes in sizes: one counts every notation, the other every meaning — and is strictly bigger.

ONE BIT

A bit is one distinction. Add one and you double the states — so 34 bytes beat the universe.

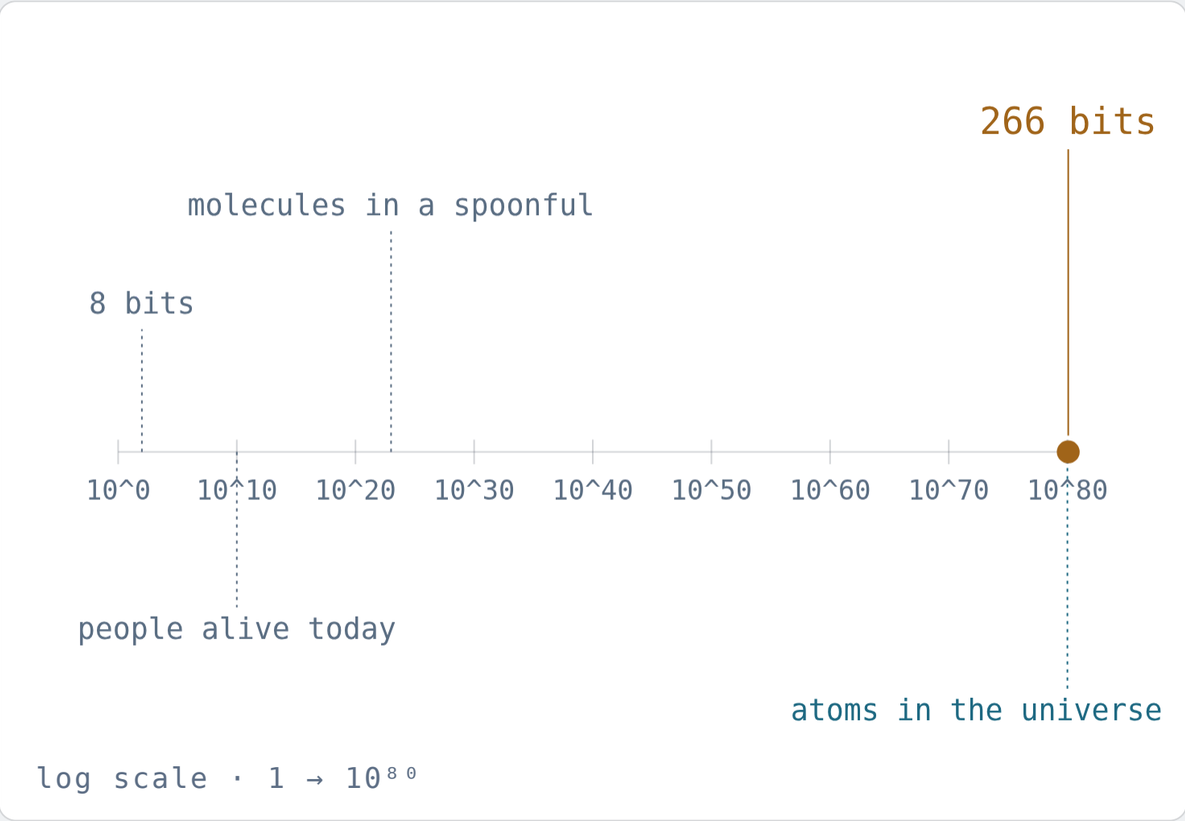
Ten bits: a thousand states. Twenty: a million. Thirty: a billion. Doubling is the most underestimated operation in human reasoning — and the entire engine of computing.

10^{80}

ATOMS IN THE OBSERVABLE UNIVERSE

2^{266}

STATES OF 266 BITS ≈ 1.2×10^{80}



A SYSTEM YOU CAN READ · ON A MACHINE YOU CANNOT

Selfie is 12,394 lines. Its machine has $2^{34,359,738,368}$ states.

Selfie is the specimen this talk uses: one file of C, containing a compiler, an emulator and a hypervisor, each of which is applied to *itself*. Small enough to read to the end — which turns out to be the only way a meaning is ever actually pinned down.

The machine it runs on has 4 GB of memory — 2^{35} bits. Written out in decimal, its state count has about 10.3 billion digits: 3.4 million pages, some 6,900 volumes of 500 pages. That shelf does not hold the states. It holds *the number* of them, written down once.

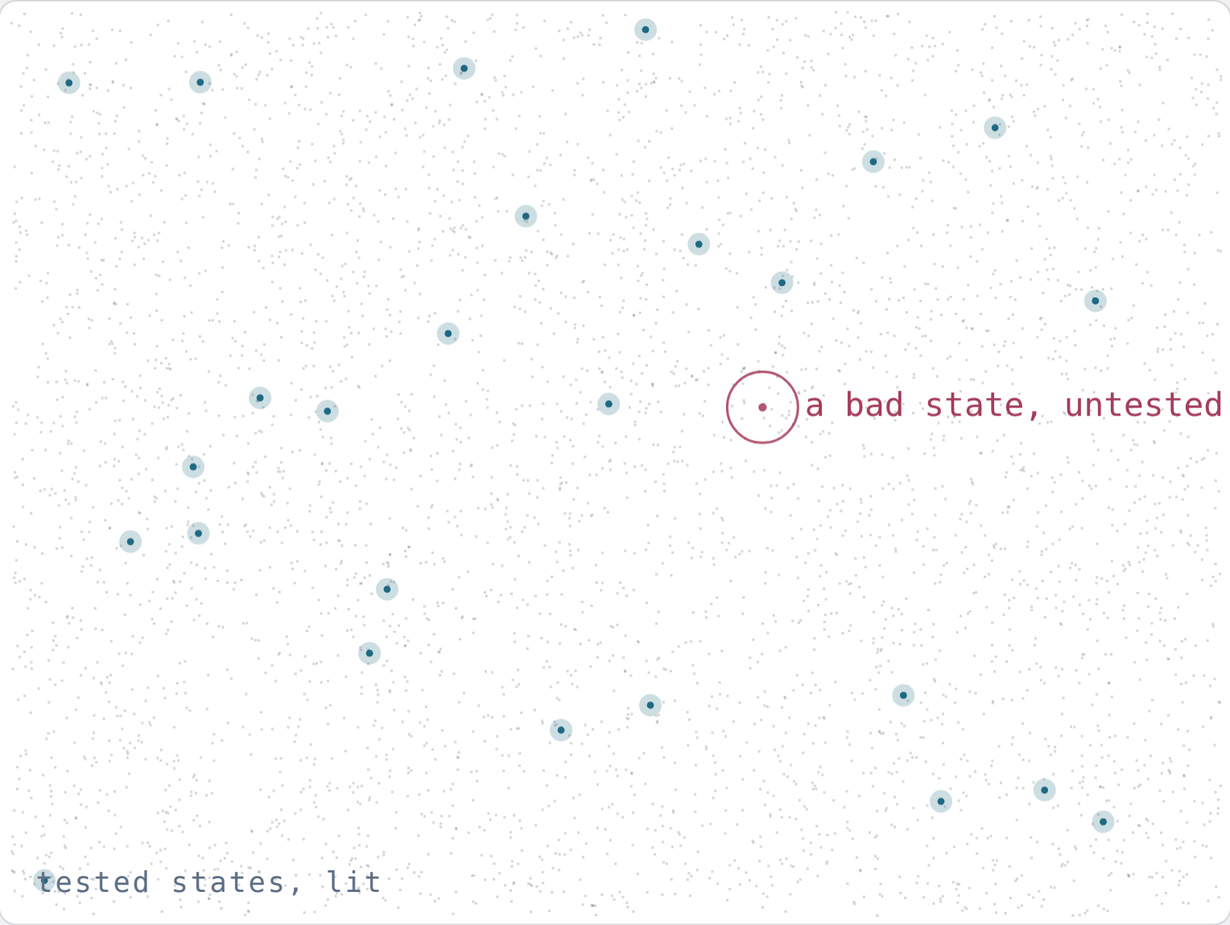
Every program selfie will ever run, and every bug in any of them, is one point in that space.

```
$ ./selfie -c selfie.c
selfie compiling selfie.c to 64-bit RISC-U with 64-bit starc
365784 characters read in 12394 lines and 1741 comments
491 global variables, 661 procedures, 512 string literals
188392 bytes generated with 43492 instructions
```

```
2^34359738368 = 23873117371686870504051737186753437903168979747538053465075696759119864115092050339869941054292537183290022099175144573637279641592487667345504934445665896266534390508060257364588787358720564536368615392783093371704721401685574994838137187636205568269232431289180378602417641004581755214355954939983542798793703970552420972448293831600998200108824043991385179964603310
```

THE CONSEQUENCE

In a space that big, good states and bad states look alike.



Test a billion states per second, starting at the Big Bang, and by now you would have checked about 2^{88} of them — out of $2^{34,359,738,368}$.

The fraction is not small. It is indistinguishable from zero.

> *Testing shows the presence, not the absence of bugs.*
>
> EDSGER W. DIJKSTRA, 1969

So knowing what a program does cannot be exhaustive search. It has to be argument, structure, abstraction — which is to say, notation. That is where Part III begins.

PART II

Infinity

Vast is still finite. Now the things that are not — and the discovery that endlessness comes in two sizes.

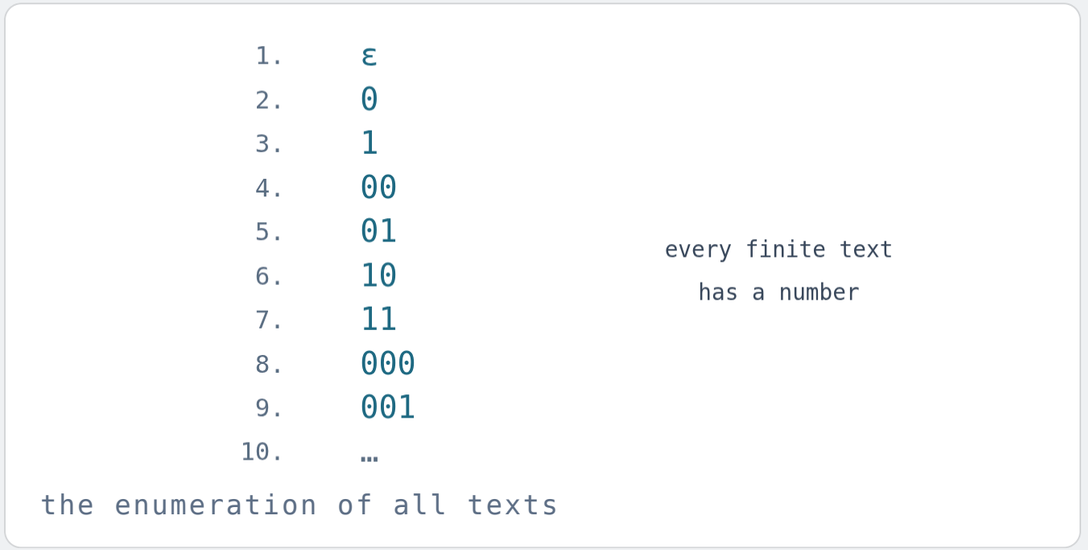
NOTATION

Everything you can write down is countable.

Two sets are the same size if you can pair them up, one to one, nothing left over. A set pairable with `1, 2, 3, …` is countable: it can be listed.

A program is a finite string of symbols. So is a proof, a specification, a prompt. List all strings of length 1, then 2, then 3 … every finite text appears at some finite position.

So all programs that will ever exist form one list. Same for all proofs. Same for all prompts anyone will ever type.

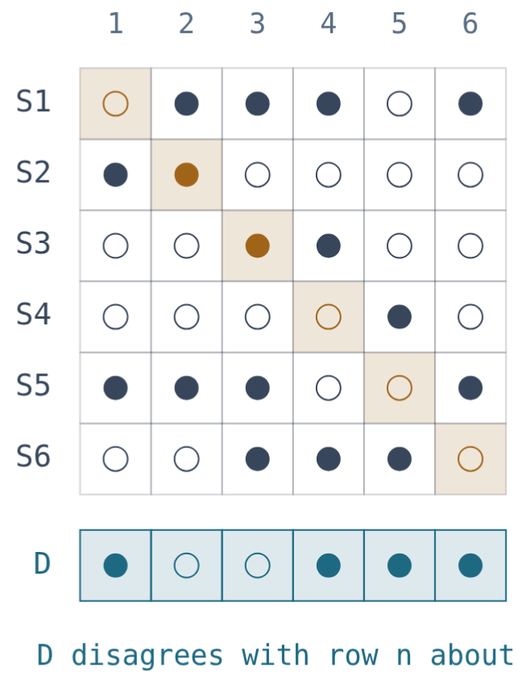


THIS HAS A NAME

Giving each piece of text a number is *Gödelisierung* — Gödel numbering. `selfie.c` is 365,784 characters: one string, one very long number. It is how a machine reads a machine, and Part III is built on it.

THE PROOF

Hand me a list of every behaviour. I will build one that is not on it.



A behaviour is an endless answer sheet: for input 1, yes or no; for input 2, yes or no; forever. Every infinite bit string is one. Suppose they could be listed: `S1, S2, S3, …`, every behaviour somewhere on the list.

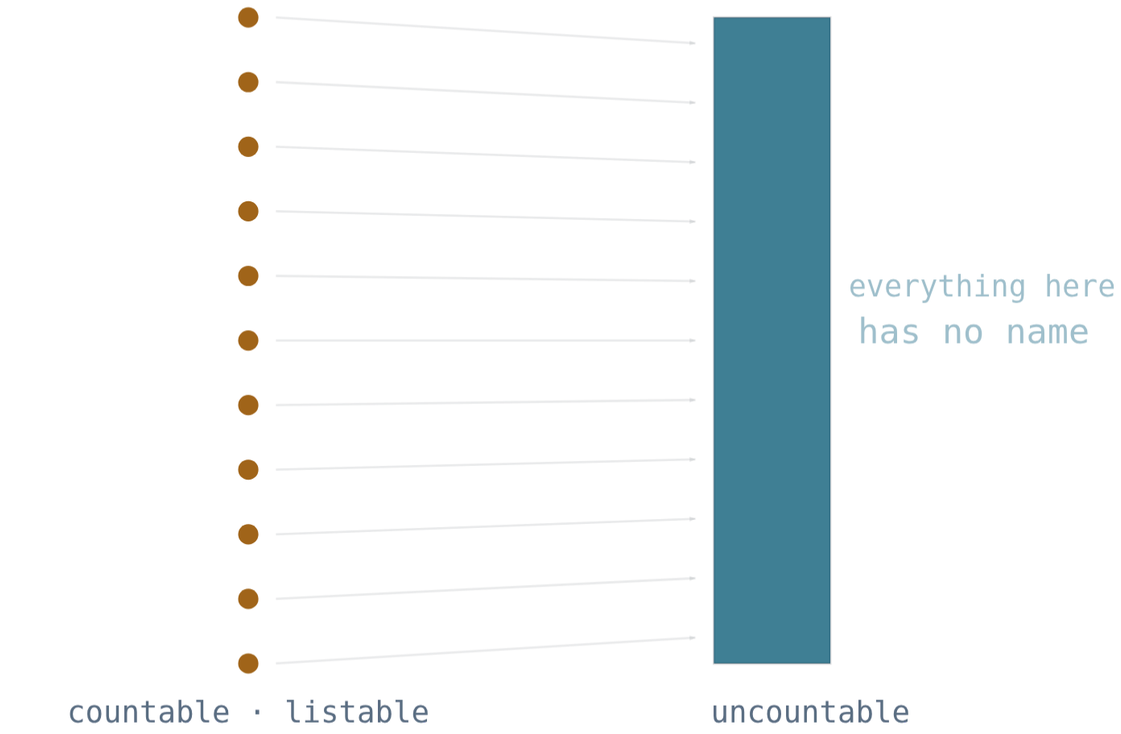
Go down the diagonal and ask each row about *its own number*. What does `S1` say about 1? `S2` about 2? `Sn` about `n`?

Build `D` by answering the opposite every time: `D(n) = not Sn(n)`. Nothing exotic — a rule anyone can apply, one number at a time.

`D` is not `S1`: they disagree about 1. Not `S2`: they disagree about 2. Not `Sn`, for any `n`. So the list left something out — and it was any list at all. Cantor, 1891.

THE RESULT EVERYTHING HANGS ON

There are incomparably more meanings than notations.



Programs: countable. Behaviours: uncountable. So almost every behaviour has no program.

Proofs: countable. Truths about numbers: uncountable. So almost every truth has no proof.

Prompts: countable. Things you might mean by one: uncountable. So almost everything is unsaid.

TURN IT OVER

No vocabulary — mathematical, legal, neural — will ever cover the space of meanings. Every new notation captures meaning that was unreachable before, and there is always more left. The supply never runs out.

PART III

Self-Reference

Counting says the gap exists. Self-reference walks to the edge and points — and the same step, taken forwards, builds the computer.

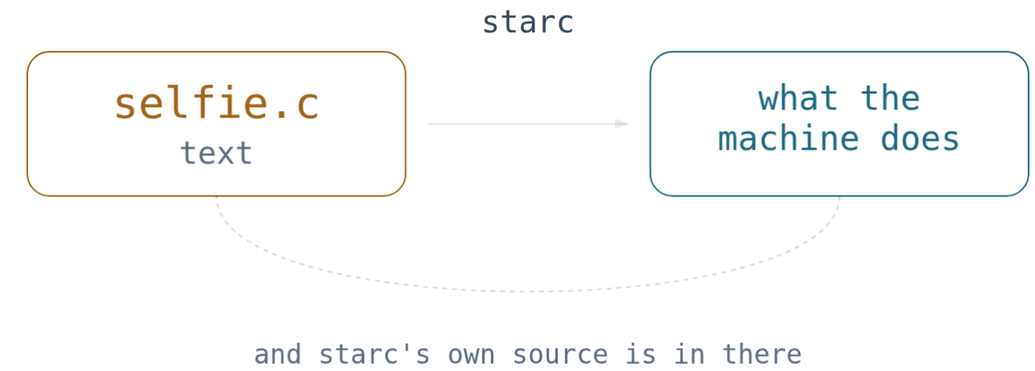
SYNTAX AND SEMANTICS

A compiler defines the meaning of the language it is written in.

What does `while` mean? Not what the manual says. What it means is whatever the compiler *emits* — a comparison and a branch — and whatever the machine then does with those.

So the meaning of a program is fixed by another program. And selfie's compiler, starc, is written in the very language it gives meaning to.

An English dictionary written in English. Only paradoxical if you demand that the definitions come first. They do not: the machine comes first, and the dictionary is bootstrapped onto it.



notation → meaning

THE DISTINCTION, IN THE ROOM'S OWN TERMS

Syntax is the text — finite, checkable, copyable. Semantics is what the machine does with it — over all inputs, forever. A semantics is a *function* from the first to the second, and pinning that function down is the founding act of every exact discipline.

GÖDELISIERUNG · YOU CAN RUN IT

Three commands. Three kinds of self-reference.

```
// self-compilation: the compiler compiles its own source — twice — and the results agree
$ ./selfie -c selfie.c -o selfie1.m -m 2 -c selfie.c -o selfie2.m
$ diff -s selfie1.m selfie2.m
Files selfie1.m and selfie2.m are identical
```

```
// self-execution: the emulator executes its own machine code
$ ./selfie -c selfie.c -o selfie.m -m 2 -l selfie.m -m 1
```

```
// self-hosting: the hypervisor hosts a virtual machine running the hypervisor
$ ./selfie -c selfie.c -o selfie.m -m 3 -l selfie.m -y 2 -l selfie.m -y 1
```

The first is Gödel's move: a text handed to itself. The second is Turing's: a machine handed its own description. The third is what an operating system is. All three run on a laptop.

THEOREM

There are truths with no proof — and no strong system can certify itself.

GÖDEL · INCOMPLETENESS · 1931

> *Any consistent formal system rich enough to describe arithmetic contains statements that are true but not provable within it. And it cannot prove its own consistency.*

HOW · THE SAME DIAGONAL

By Gödelisierung, build a sentence that says *"this sentence has no proof in this system."* If it were provable, the system proves a falsehood. So it is unprovable — and therefore true. Adding it as an axiom does not help: the construction runs again.

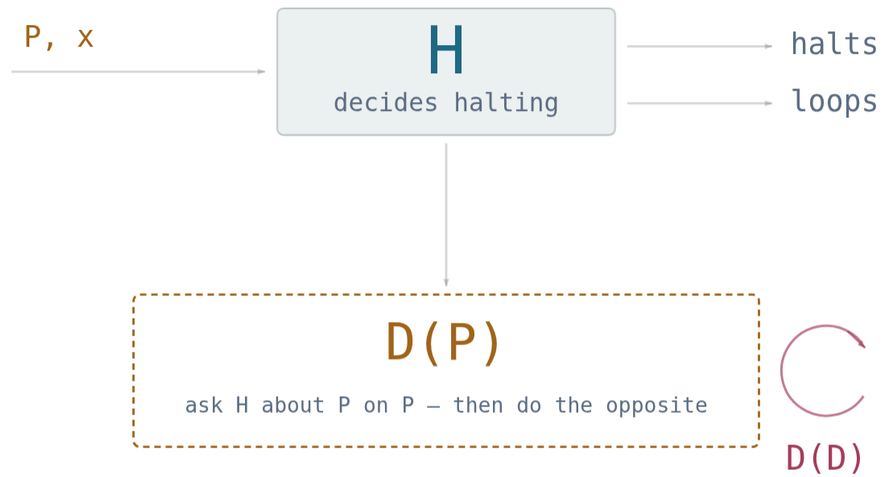
READ IT AS ENGINEERING · THOMPSON 1984

Selfie's fixed point proves the compiler agrees with *itself*. It cannot prove the binary does what the source says: a compiler can carry a back door that reinserts itself on every self-compilation, invisible in clean source. The check has to come from outside — a second, independent compiler.

Proof is a finite object you can check. Truth is not. Proof is syntax. Truth is semantics. They are not the same size — and nothing certifies itself.

UNDECIDABILITY

Will this program ever stop? No program can always tell.



halts ⟺ ¬halts — so H cannot exist

feed the decider its own description

Suppose `H(P,x)` decides, for every program and input, whether `P` halts on `x`.

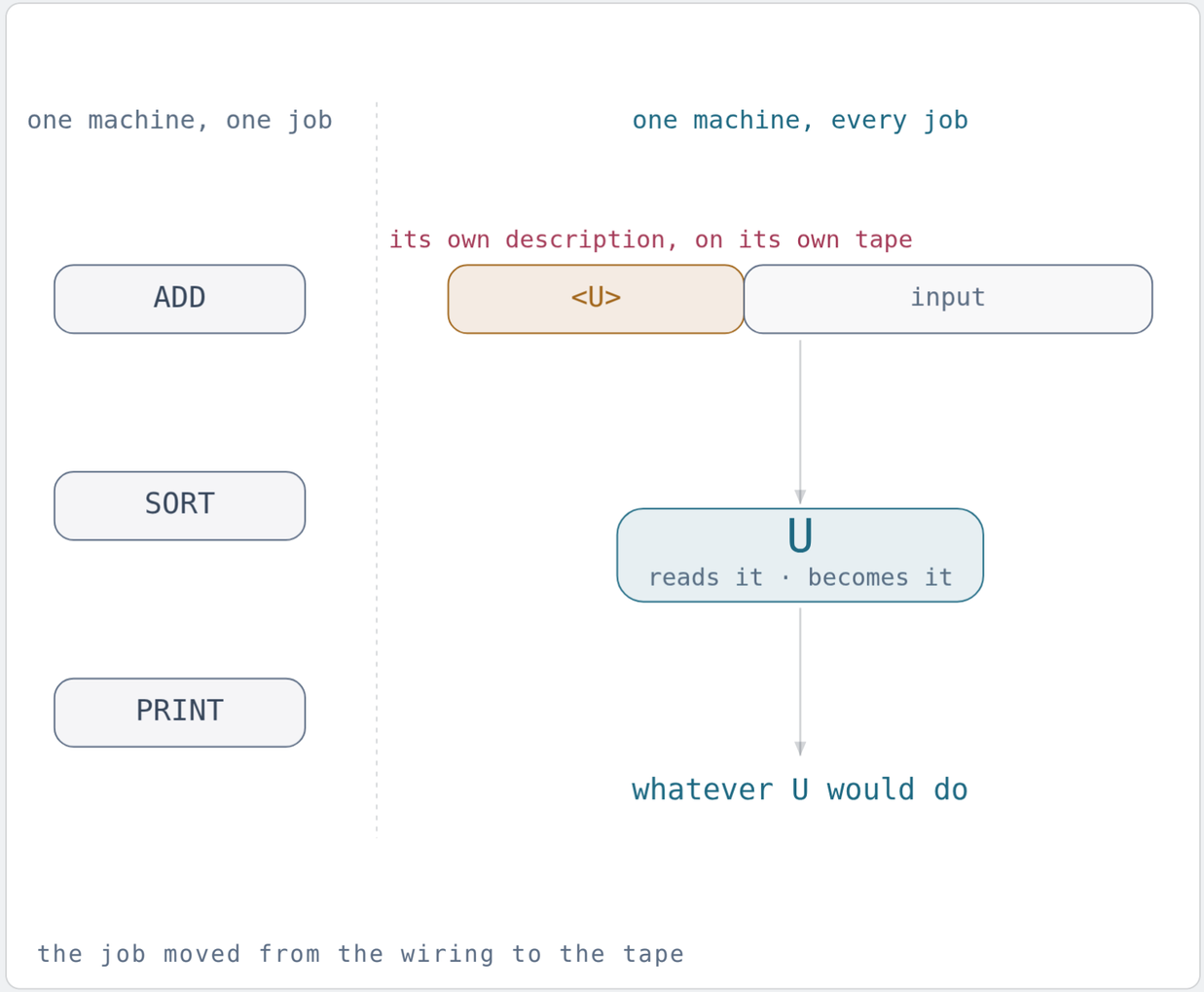
Build `D(P)`: ask `H` whether `P` halts on `P` — then do the opposite.

Now run `D(D)`. It halts exactly if it doesn't. Contradiction. So `H` never existed.

Turing, 1936 — the same paper that defined the universal machine, and thus invented the computer. The limit and the machine arrived together.

THE SAME MOVE, FORWARDS

One machine that can be any machine.



A machine used to *be* its job. Turing's move: put the machine's description *on the tape*, as data. Then one machine U reads any description and does whatever that machine would do. That is universality: one piece of hardware, every possible behaviour, because the behaviour arrives as notation.

Your phone is not phone-shaped. It is U holding a description — which is why a new app needs no new hardware.

SELFIE'S U IS A PAGE OF C PER INSTRUCTION GROUP

mipster reads a word of RISC-U code, does what it says, and advances. Fourteen instructions, and it runs every program that will ever exist for that machine — including itself. That is the second of the three commands.

ONE SENTENCE, READ TWICE

Hand a decider its own description: contradiction, no H. Hand a machine *any* description: every program at once, one U. Turing published both in 1936.

THE GENERAL CASE

It is not just halting. It is every interesting question about meaning.

> RICE · 1953
>
> *Every non-trivial property of the behaviour of programs is undecidable. Only questions about their text are safe.*

DECIDABLE · ABOUT SYNTAX

Does it parse? How long is it? Which procedures does it call? Are the types consistent? Does it contain a loop? A compiler answers all of these, always, in finite time.

UNDECIDABLE · ABOUT SEMANTICS

Does it terminate? Is it equivalent to that one? Can it divide by zero? Does it leak the key? Is it correct? No program decides any of these for all programs — whoever, or whatever, wrote them.

So every useful tool — a type checker, a test suite, a fuzzer, a model checker, a proof assistant — is a deliberate approximation: sound but incomplete, or complete but unsound, or exact only within a bound. Choosing which to give up *is* the discipline. Selfie ships several such choices, applied to selfie.

PATTERN

One idea. Sixty years. Six theorems.

YEAR	WHO	THE LIST	THE DIAGONAL OBJECT
1891	Cantor	all behaviours	a row on no list
1901	Russell	all sets	the set of non-self-members
1931	Gödel	all provable sentences	"I am unprovable"
1936	Tarski	all definable predicates	"I am false"
1936	Turing	all decidable questions	a program that defies its judge
1953	Rice	all semantic properties	all of the above, at once

Assume a complete list. Ask each item about *itself*. Answer the opposite.

Learn the move once and you own the century.

PART IV

Cost

Suppose a question *is* decidable. You still have to pay for the answer — in time, space, and energy.

DECIDABLE ≠ DOABLE

A question with a guaranteed answer you will never receive.

Given a logical formula over 100 yes/no variables: is there an assignment making it true? Perfectly decidable — try all 2^{100}.

2^{100}

≈ 1.3×10^{30} ASSIGNMENTS

10^{13} years

AT A BILLION CHECKS PER SECOND

WHERE SELFIE MEETS IT

monster and rotor translate RISC-U machine code — including all of selfie — into exactly such formulae, satisfiable when some input makes the code fail. Syntax becomes semantics, and then hits this wall, before the coffee gets cold.

INTRACTABILITY

Hard to find. Easy to check.

That asymmetry has a name: NP — problems whose solutions are quick to verify even if finding them seems to need exponential search.

Cook and Levin, 1971–73: thousands of such problems are *the same problem* in disguise. Crack one efficiently and you crack scheduling, routing, folding, packing, proving.

Whether that is possible — `P = NP?` — is the most consequential open question in the exact sciences. Most researchers bet no.

THE ASYMMETRY, EVERYWHERE

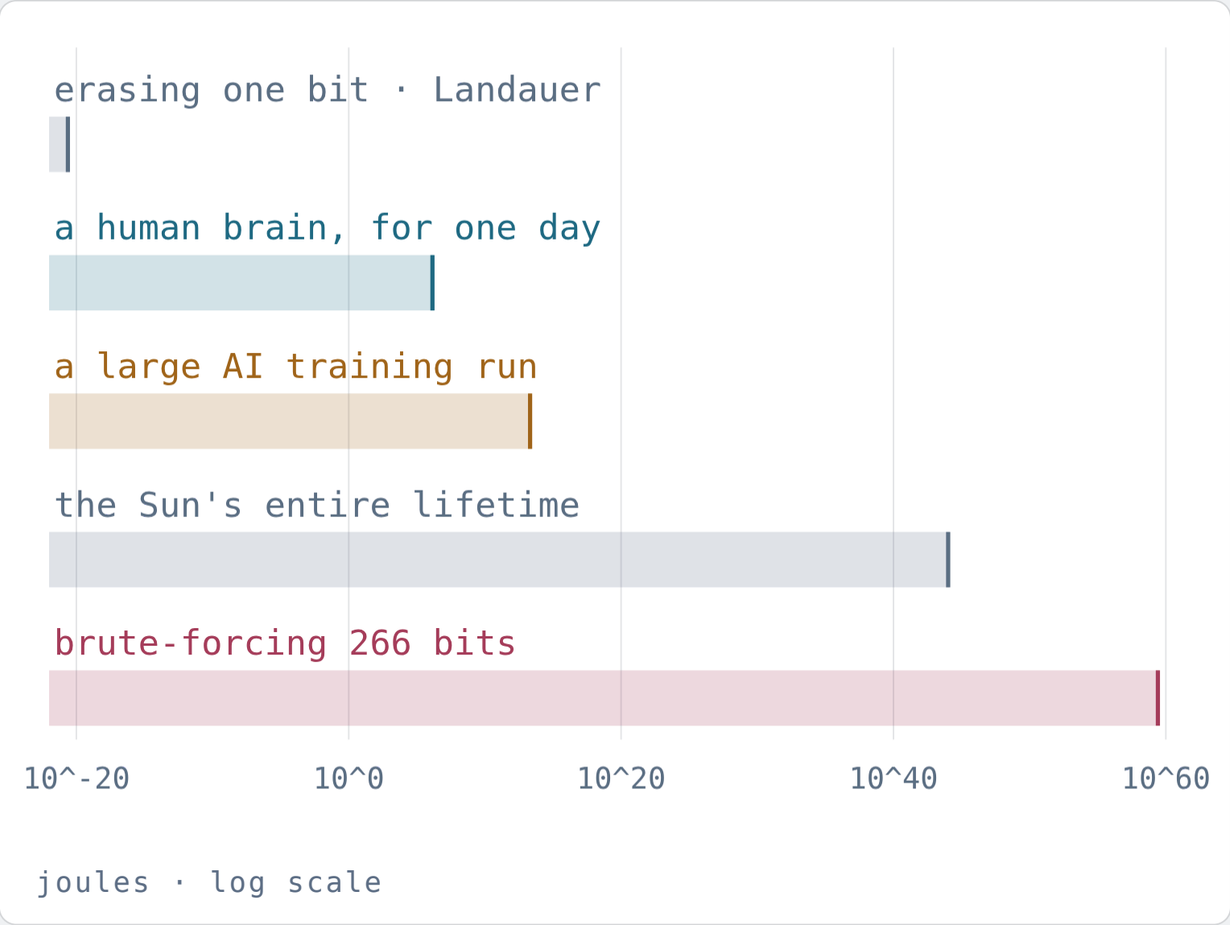
Writing a proof versus reading it. Designing a protein versus assaying it. Writing the program versus running the test. Doing the homework versus grading it.

THE ASYMMETRY, NAMED

Generation is expensive; verification is cheap. Every division of labour that works — between people, or between a person and a machine — sits on that gap. Part V comes back to it.

PHYSICS

Computation is not abstract. It costs energy, by law.



Landauer, 1961: erasing one bit at room temperature costs at least $kT \ln 2 \approx 3 \times 10^{-21}$ joules. Information is physical.

So brute-forcing our 266-bit space costs $\approx 10^{59}$ J — about 10^{15} times everything the Sun will radiate in its entire lifetime.

Meanwhile a human brain runs on 20 watts: a dim light bulb, doing what data centres cannot.

Time, space, energy — three currencies for one budget. Any claim about intelligence that ignores the budget is a claim about magic.

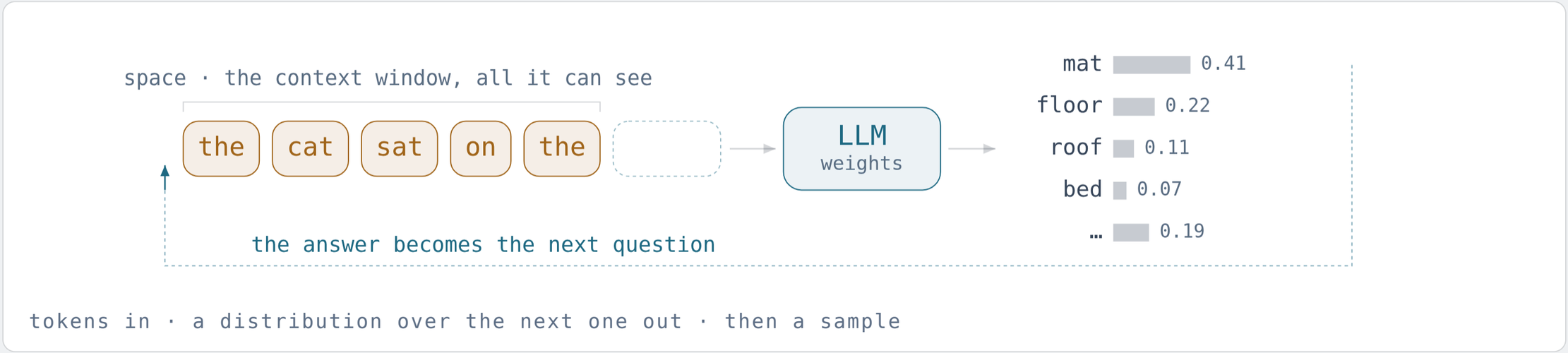
PART V

Machines

Now the machine everyone is asking about. It is a computation — so it is subject to every single thing we have just established.

WHAT IT IS

A large language model is a machine trained on notation, asked for meaning.



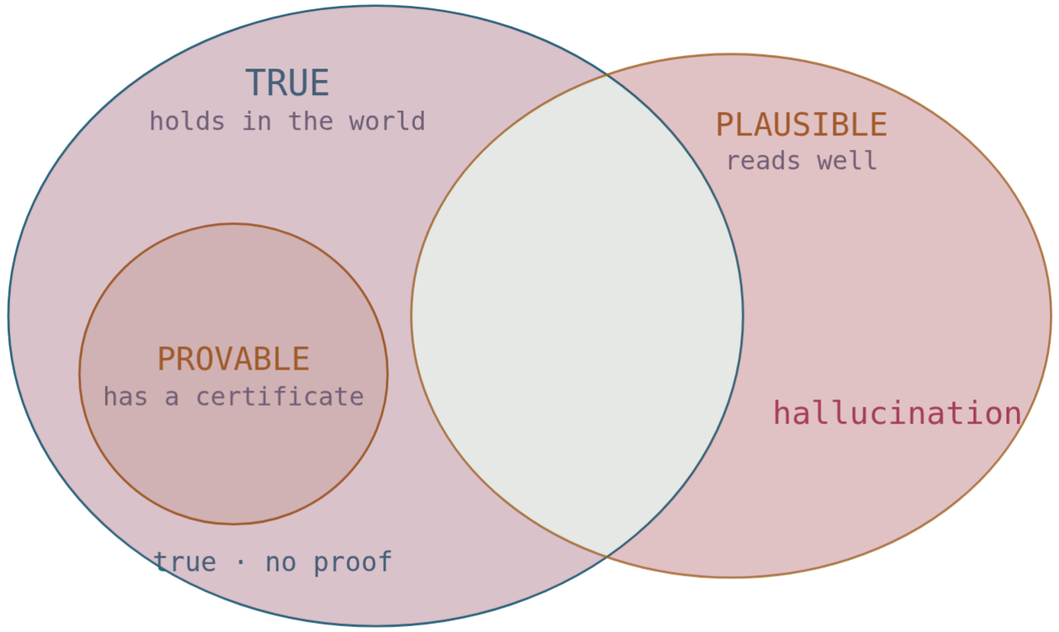
The name is exact. What a large language model — an LLM — is trained on is *language*, so the signal is syntactic: which symbol follows which. Meaning is never handed to it.

That this works as well as it does is the genuine surprise of the decade. It cost megawatts for weeks; you run on twenty watts. Neither figure changes what kind of object it is.

Billions of parameters, each many bits. Part I applies unchanged: that space cannot be inspected, so the behaviour cannot be enumerated. Only sampled.

THE OLD GAP, INDUSTRIALISED

A hallucination is a proof-shaped object that isn't true.



three tests · and they are not the same test

Fluency is syntax. Correctness is semantics. We built a machine of extraordinary fluency, so the gap between the two is something you now meet before breakfast.

Not a defect to be patched away: it is the proof/truth distinction at consumer scale. Generated code compiles, reads well, and is wrong in the same way a proof-shaped paragraph is wrong.

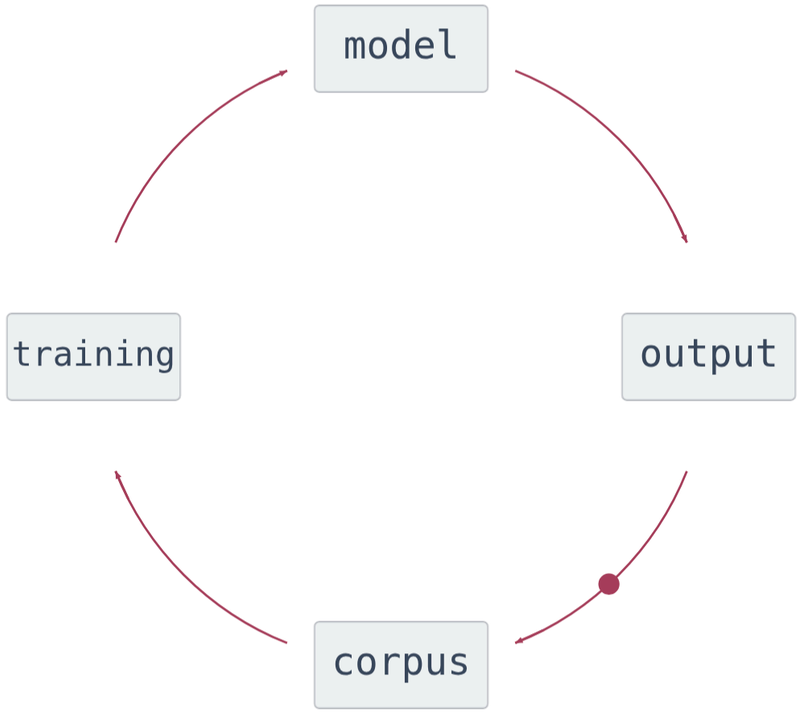
So the only thing that settles it is a check against something with a semantics — a compiler, a test, a proof, a measurement, an experiment.

WHERE THE SEMANTICS COMES FROM

Not from the model: it was trained on symbols defined by other symbols. From outside it. Part III already said the check is always external — Gödel's stronger system, Thompson's second compiler. Here it is, at consumer scale.

WHAT ACTUALLY CHANGED

Generation got cheap. Verification did not.



systems judging, training, writing systems

— Producing a program, a proof sketch, a draft was expensive and therefore scarce. Scarcity did the filtering. Now production is nearly free, and Part IV's asymmetry is exposed: the checking side did not move.

— Models are trained on text that models wrote. Models grade models. Models write the code that trains models. The judge and the candidate share the same blind spots.

— "Is this code correct?" is a semantic property. Rice: no general decision procedure. "Is this system right about itself?" Gödel: no self-certificate. An explanation of its own output is more output.

— So the two ends of the loop are not generation: saying what should be true and establishing that it is. Both are acts of meaning, and neither is more notation.

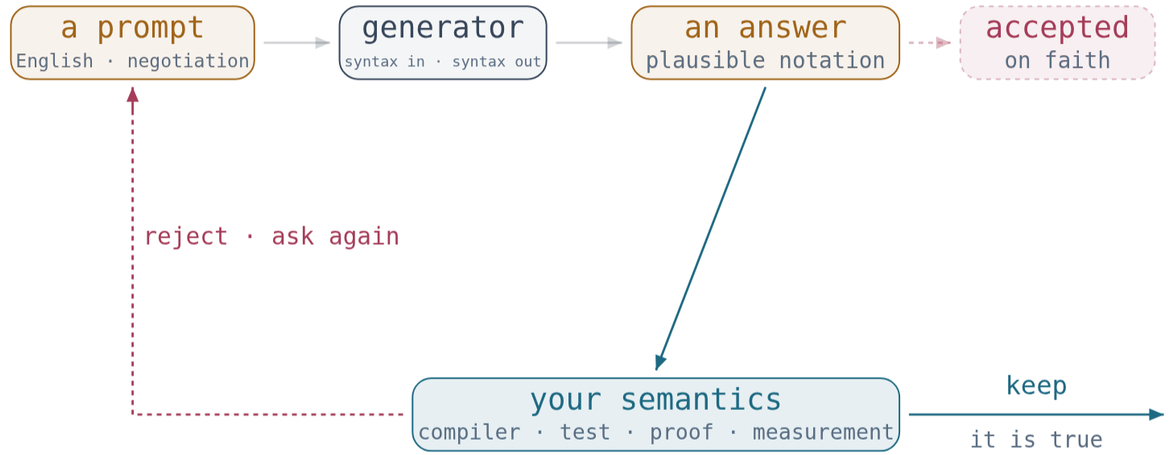
PART VI

Definition

Everything so far was true before the machine arrived and stays true after it. Now the answer to the title — and a specimen to run.

THE LOOP

A generator supplies notation. The semantics has to come from somewhere else.



what the machine supplies · and what it does not

Prompt in English, and what comes back is plausible notation: fluent, confident, and Part V says nothing about whether it is true.

Two things can happen next. Unchecked, the answer is simply accepted — and then there is no semantics anywhere in the loop.

Checked, it meets a compiler, a test, a proof, a measurement — a semantics in Part III's exact sense — and is kept or sent back round. That gate is the one part of the loop the machine does not supply.

> THE SAME SHAPE, AGAIN
>
> *Cantor's list needed an object built from outside it. Gödel's system needed a stronger one. Thompson's compiler needed a second compiler. The check is external every time — and what supplies it is a domain: languages, and their semantics, understood.*

THE DEFINITION

So — what is computer science?

WORKING DEFINITION

Computer science is the exact study of notation a machine can execute — what can be written down, what can be computed, what can be decided, and what can be afforded — and therefore of the gap, measured precisely, between notation and meaning.

NOT ABOUT COMPUTERS

The machine is where notation is forced to be exact, so the gap was noticed here first and measured here most sharply. But the theorems hold for any notation: a statute, a genome, a score, a model.

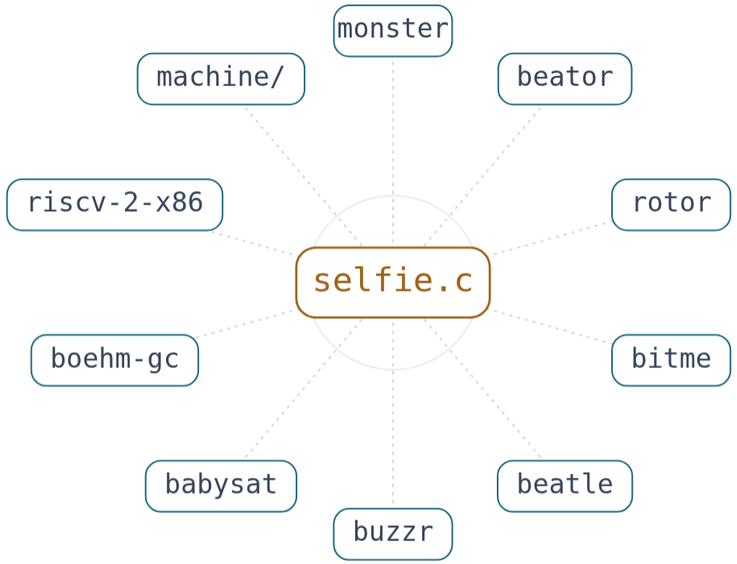
AND NOT FINISHABLE

No final language (Cantor), no complete system (Gödel), no self-certificate, no decision procedure for meaning (Rice). The subject cannot be completed — a property of the subject, not a complaint about it.

Test it: a compiler, an operating system, a proof assistant, a language model — each is notation given a meaning by a machine, and each is bounded by Parts I to IV. None of them is outside the definition.

THE SPECIMEN

One file that is the whole definition, small enough to read to the end.



built on selfie · applied to selfie

A LANGUAGE, AND ITS MEANING

C*: 7 keywords, one type and pointers to it. starc gives it a semantics — and starc is written in C*, so the dictionary is written in the language it defines.

A MACHINE

RISC-U: 14 instructions, 32 registers, 4 GB. mipster interprets it — one universal machine, a page of C per instruction group, running every program for that machine, itself included.

A FIXED POINT, AND ITS LIMIT

Selfie compiles itself and gets the same bytes twice — proof that it agrees with itself, and Thompson says what that cannot prove. So the tools around it turn its execution into logic, and are applied to selfie and to themselves: the outside checks, built in.

REFRAME

Truth can only be approximated.
The approximating never stops.

WHAT THE THEOREMS FORBID

A final language. A complete system. A machine that certifies itself. A decision procedure for meaning. A search that reaches every state, or an energy budget that could pay for one.

WHAT THEY LEAVE OPEN

Every notation not yet invented, every truth not yet proved, every behaviour not yet written down — an unbounded frontier in a subject that, by its own theorems, cannot be finished.

The halting problem, applied to the subject itself:
this computation does not terminate.

CLOSE

Notation is finite. Meaning is not. The gap is the subject.

Computer science is the field that measured it — not to close it, which four theorems say cannot be done, but to say exactly where it is.

That is the whole answer. One slide left: the sources, and the specimen, ready to run.

SOURCES · AND THE SPECIMEN

Further reading — and the file itself.

THE THEOREMS

Cantor 1891 · Russell 1901 · Gödel 1931 · Tarski 1936 · Turing 1936 · Rice 1953 · Cook 1971 · Karp 1972 · Levin 1973

PHYSICS AND ENGINEERING

Landauer 1961 · Dijkstra 1969 · Thompson, *Reflections on Trusting Trust* 1984 · Wheeler 2005

THE SPECIMEN

```
$ git clone github.com/cksystemsteaching/selfie
$ cd selfie && make
$ ./selfie -c selfie.c -m 2 -c selfie.c
```

Or `docker run -it cksystemsteaching/selfie`. It also runs in a browser tab.

THE LONG VERSIONS

`/intelligence` — the same argument, an hour long, for any field. `/talk` — the three commands, explained.

`selfie.cs.uni-salzburg.at`

BOOK

Kirsch, *Elementary Computer Science: From Bits and Bytes to the Universality of Computing* — built on the specimen.

`github.com/cksystemsteaching/selfie`